CSE465 Report

SPRING 2026 - 14 May 2026

Multilingual Speech-to-Speech Duplex Voice Translation: Layer Pruning and Knowledge Distillation over Large S2S Translation Models to Tiny On-Device Model with Duplex Voice Translation support

Md. Tanvir Chowdhury
ID: 2232122042
ECE Department
North South University
tanvir.chowdhury01@northsouth.edu

Rayed Riasat Rabbi
ID: 2311649042
ECE Department
North South University
rayed.rabbi@northsouth.edu

Nazmus Sakib Nihal
ID: 2311748042
ECE Department
North South University
nazmus.nihal@northsouth.edu

Abstract

This project studies practical compression of Meta's SeamlessM4T v2 Large speech-to-speech translation model for lower-resource multilingual deployment. SeamlessM4T v2 Large is a powerful multilingual speech translation system, but the speech-to-speech variant used in this project has approximately 1.805B parameters and multiple coupled components: a speech encoder, text decoder, text-to-unit (T2U) model, and vocoder. The project goal was to reduce parameter count and inference cost while preserving usable speech translation quality for English, Bengali, Mandarin, Arabic, and Hindi.

The final approach combines language-scoped vocabulary pruning, Block-Influence-guided depth pruning, conservative T2U layer merging, text decoder pruning, checkpointed recovery fine-tuning, sparse top-k teacher-student knowledge distillation, and LoRA/DoRA-style adapter recovery. The final selected multilingual model has approximately 1.0879B parameters, a 39.7 percent reduction from the baseline. On the saved eight-direction multilingual ASR benchmark, it achieves 33.73 ChrF, 8.16 BLEU, and 0.1336 real-time factor (RTF), compared with the 1.8055B baseline's 46.49 ChrF, 15.88 BLEU, and 0.2455 RTF. This gives a 1.84x RTF speedup and makes the model practical on a local RTX 3050 Laptop GPU with 4.3 GB VRAM.

1 Introduction

Speech-to-speech translation is valuable when users need direct spoken communication across languages without reading an intermediate transcript. However, modern multilingual speech translation systems are large. SeamlessM4T v2 Large is designed for broad language coverage and high quality, not for constrained deployment. Running it on a

consumer laptop GPU, a classroom demo machine, or a lightweight web backend requires compression.

Can a large multilingual speech-to-speech model be structurally pruned enough to run in a practical local/web setting while still producing intelligible multilingual speech translation? The project focused on five languages: English (eng), Bengali (ben), Mandarin Chinese (cmn), Arabic (arb), and Hindi (hin).

The final model is not just a notebook result. The work also includes a local inference notebook tested on an NVIDIA GeForce RTX 3050 Laptop GPU, a duplex browser web app using FastAPI, WebSockets, browser AudioWorklets, boundary detection, and streamed audio playback, along with a documented failed sub-500M branch which reveals a real lower bound of structural pruning for this architecture.

2 Literature Review

The Seamless paper introduced a family of multilingual expressive and streaming speech translation systems, including SeamlessM4T v2. The architecture is a pipeline with a speech encoder, text generation stage, unit generation stage, and vocoder.

Ushio, Zhou, and Camacho-Collados proposed vocabulary trimming for multilingual language model compression, an idea adapted here to reduce the embedding matrices for the targeted five languages. ShortGPT studies Block Influence (BI) based on cosine similarity to identify redundant layers, which this project utilized as a pre-filter for layer removal.

Recent machine translation compression work, including CULL-MT and iterative layer pruning approaches, motivates structural pruning of selected language directions and transformer layers iteratively. Conversely, FLAP proposes structured width pruning; however, this project tested and found that FLAP-style width pruning caused catastrophic generation quality collapse after depth pruning.

For recovery, DPHuBERT combines distillation and pruning for self-supervised speech models, inspiring this project's structural compression followed by teacher-student recovery. DoRA and LoRA-style parameter-efficient fine-tuning were adopted to approximate full fine-tuning behavior with fewer trainable parameters. Finally, the MMS project scales speech recognition and synthesis, and MMS ASR was utilized for evaluation where audio output quality mattered.

3 Proposed Methodology

The task is multilingual speech-to-speech translation. To achieve an efficient, deployable model without catastrophic degradation, we utilized a multi-phase structural pruning and recovery pipeline. The complete methodology is detailed below:

3.1 Phase 0: Baseline capture

The optimization process began by establishing a foundational baseline using the facebook/seamless-m4t-v2-large model initialized in a speech-to-speech configuration. It was evaluated against the multilingual FLEURS dataset to concretely define the initial quality and speed targets before any structural modifications were introduced.

3.2 Phase 1: Five-language vocabulary pruning

To achieve the largest low-risk reduction in parameter count, the model's expansive multilingual vocabulary was systematically trimmed down to a strict five-language target set. This was accomplished by scanning specific corpora, isolating observed token IDs alongside essential special tokens, and completely rebuilding the shared embeddings, text decoder, and language model head to map exclusively to this newly constrained vocabulary.

3.3 Phase 2: Speech encoder pruning from 24 to 16 layers

The architecture's speech encoder underwent aggressive depth reduction utilizing a block-importance (BI) guided iterative loop across calibration samples. By shielding the critical first, middle, and last layers, the process safely identified and stripped away the most redundant layers, which yielded a substantial acceleration in inference speed and significantly lowered the real-time factor.

3.4 Phase 3: T2U LaCo/RDSC merge

Because the text-to-unit (T2U) module heavily dictates the sensitive audio generation path, a conservative layer-merging strategy was favored over outright deletion. Adjacent layers within the T2U module were merged and re-indexed only when their representational similarity satisfied a strict threshold, successfully trimming the parameter footprint while introducing virtually no additional degradation to output quality.

3.5 Phase 4: Additional speech encoder pruning from 16 to 14 layers

Returning to the BI-guided iterative evaluation, the speech encoder was squeezed further by removing two additional layers. However, because the most disposable redundancy had already been eliminated in the earlier phase, this secondary pruning step proved more costly to the model's baseline translation quality, marking the limit of easy structural savings.

3.6 Phase 5: Text decoder pruning from 24 to 14 layers

The text decoder, a massively critical component for quality retention, was forced through an aggressive 10-layer reduction using the established BI methodology. While protecting the boundary layers prevented total collapse, this deep cut caused the most severe performance cliff in the entire pipeline, representing a necessary sacrifice to push the architecture closer to a highly constrained, near-1B parameter footprint.

3.7 Phase 6: Knowledge-distillation recovery

To salvage the severe translation quality lost during the aggressive decoder pruning, a robust teacher-student training paradigm was initiated using the original full-scale model to guide the pruned student network. Utilizing low-rank adapters and a loss function combining cross-entropy with sparse KL divergence—carefully managed by remapping the

full teacher logits into the student's condensed vocabulary space—the model successfully clawed back a significant margin of its predictive accuracy.

3.8 Phase 7: Final hybrid LoRA/DoRA recovery

The recovery phase was ultimately finalized by stabilizing the network through a hybrid LoRA/DoRA setup that carefully isolated trainable parameters and utilized independent optimizer groups for stable gradient scaling. This targeted refinement resolved the remaining training bottlenecks and finalized the architecture, yielding a highly optimized, deployable on-device model capable of fast and efficient inference.

3.9 Phase 8: CIF Boundary Adapter Training and Integration

The training process begins by generating a dataset from LibriTTS audio pairs injected with artificial silence gaps. To maximize efficiency, a frozen SeamlessM4T encoder extracts 1024-dimensional hidden states from these clips. These features, alongside aligned binary labels marking speech and silence, are saved directly as PyTorch tensors.

The adapter itself is a lightweight Multi-Layer Perceptron. It reduces the 1024-dimensional input sequences through two hidden blocks of 512 and 256 dimensions, applying Layer Normalization, ReLU, and 30% Dropout. A final linear layer with a Sigmoid activation predicts the probability of speech per frame.

During training, a custom data loader pads variable-length sequences with zeros, perfectly mirroring the silence labels. The model trains for 10 epochs using Binary Cross Entropy Loss and the AdamW optimizer, with continuous validation and a final test set evaluation to ensure accuracy.

Finally, the trained adapter is integrated with the base model via a custom wrapper making it aligned with the full model, rather than working as an adapter outside the model, instead it works as a part of the model itself. It continuously scans encoder output for silence thresholds, precisely slicing the raw audio waveform into manageable chunks for the standard decoder to translate without overloading.

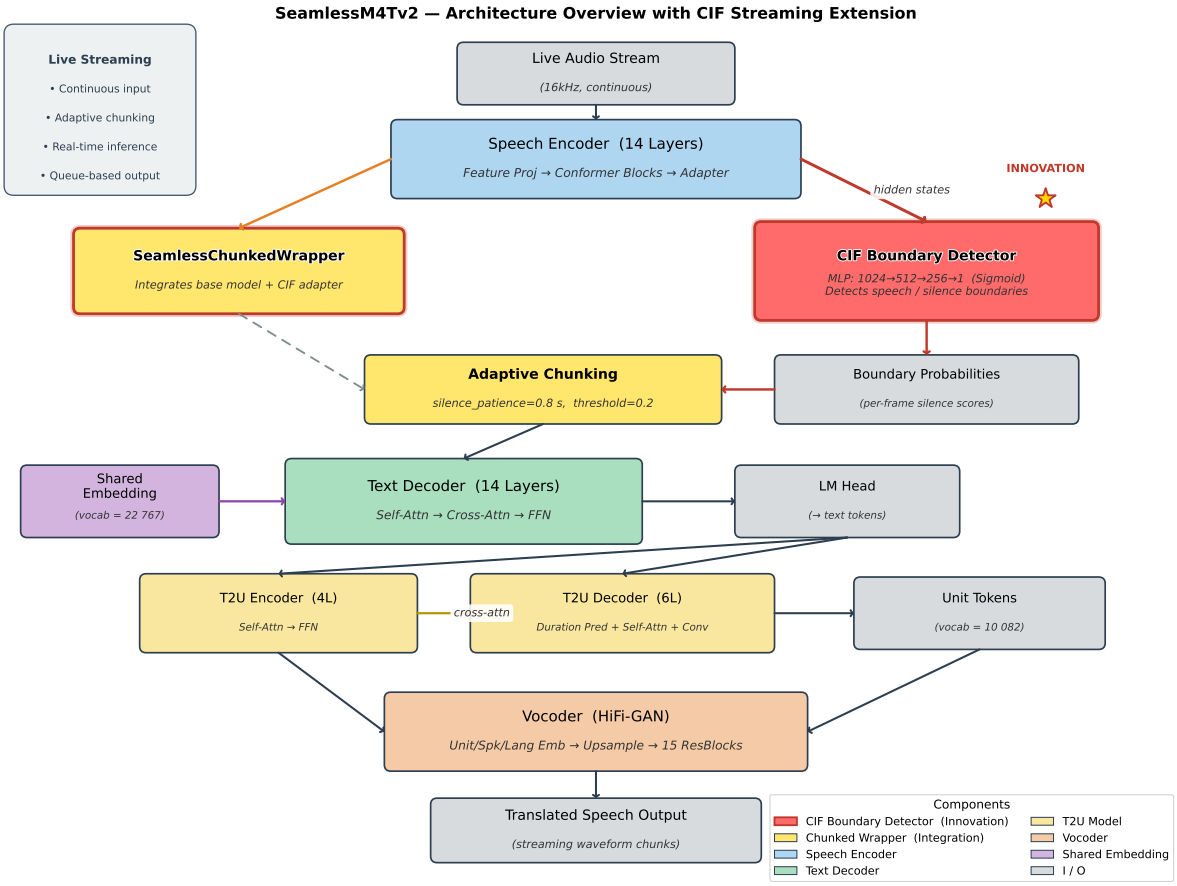

Figure 1: Overall architecture of the proposed multilingual speech-to-speech translation pipeline.

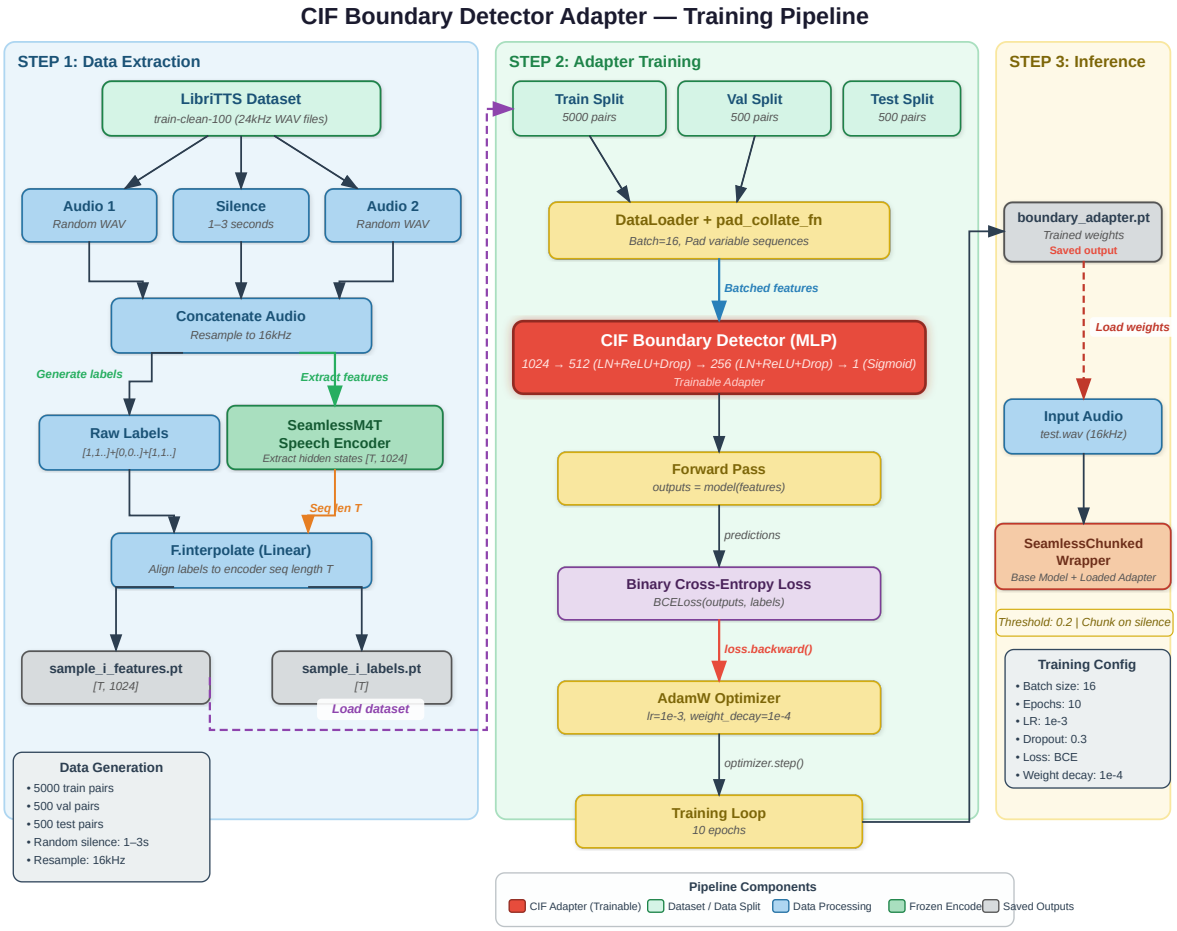

Figure 2: CIF Boundary Detector Adapter Training Complete Pipeline and Steps

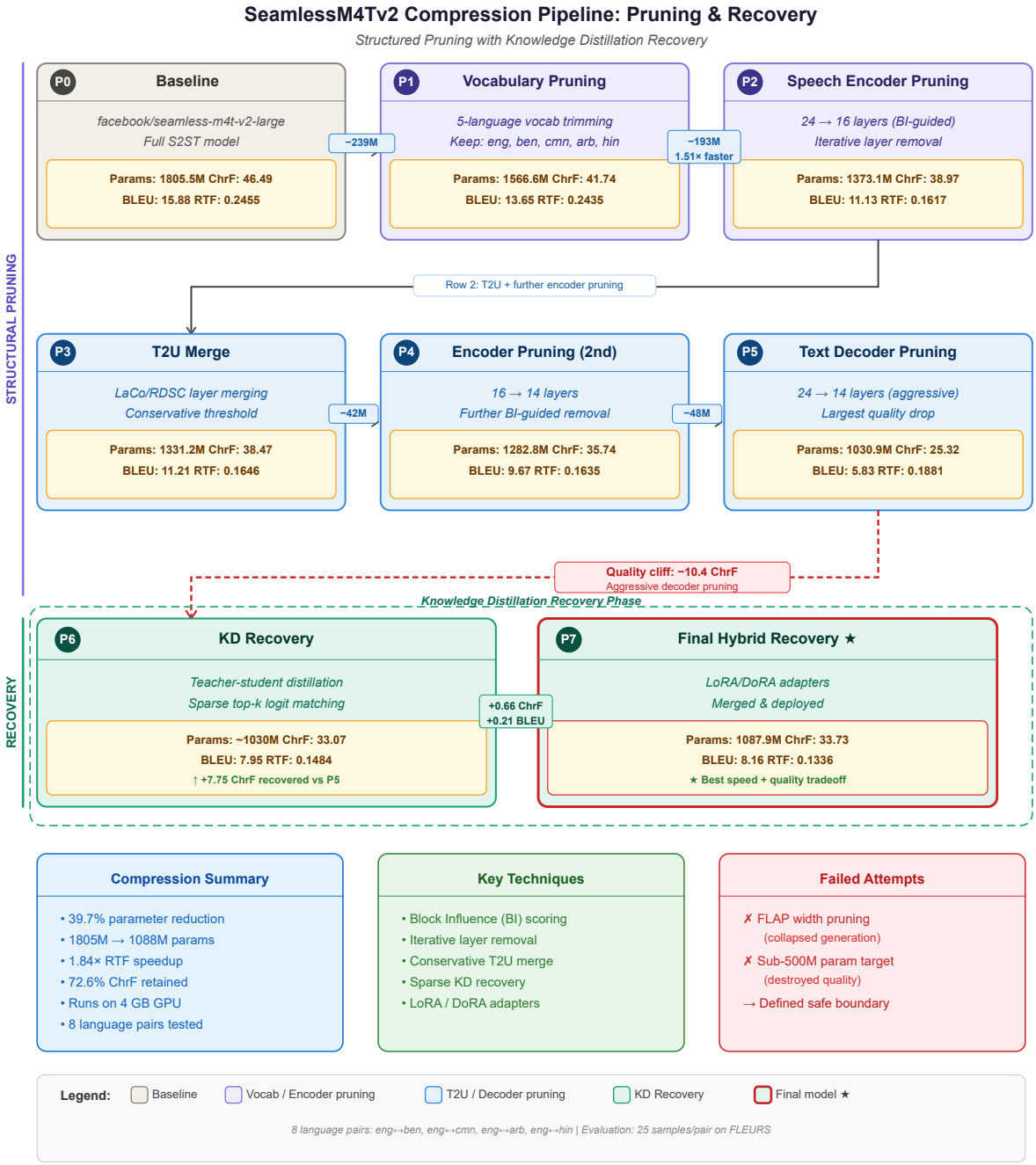

Figure 3: Complete Steps of proposed methodology using multiple layer pruning techniques

4 Experimental Setup/Experiments

Training and pruning were primarily performed in notebook environments with two NVIDIA Tesla T4 GPUs (15.6 GB VRAM each). Local inference was verified on an NVIDIA GeForce RTX 3050 Laptop GPU (4.3 GB VRAM).

The dataset utilized was FLEURS, using 25 samples per language pair across eight pairs (200 samples total). Engineering choices included Parquet paths, chunked streaming dataset loading, and language-code mapping. Software included PyTorch, HuggingFace

Transformers, Datasets, SacreBLEU, PEFT, Torchaudio, FastAPI, and WebSockets.

Alongside the proposed methodology we have also done more experiments which eventually not included in ourt proposed architecture part below:

1. We found out that pruning more than 10 layers from the speech encoder and text encoder massively destroys the models accuracy.

2. Applying FLAP width pruning somehow triggers a loop stuck, causing rtf increase and it also significantly damages the model. So FLAP is not suitable for the models which behave or architecture is similar to Seamlessm4t model

3. We tried going full textless, replace text decoder with a 50 Million parameter CIF connector. But training that to replace the text decoder didn't work out. Resulting a totally failed approach.

4. As a parallel track to the pruning-based compression pipeline, we also explored building a small speech-to-speech student model from scratch using knowledge distillation. This approach targets English-to-Chinese (EN→ZH) translation on FLEURS and serves as an alternative method for achieving a compact S2ST model. This approach was complementary to the pruning pipeline. It demonstrates that a from-scratch distilled model can achieve S2ST with a different architecture (pure transformer vs. pruned SeamlessM4T), different language pair (EN→ZH vs. EN→BN), and different synthesis backend (CosyVoice vs.SeamlessM4T vocoder). Both tracks pursue the same goal of making S2ST efficient on resource-constrained devices. But, in the end, training that model from scratch was generating poor performance matrics score, resulting we took the decision to be in our pruning technique finally.

5 Performance Metrics with Justification

- **Parameter count:** Measures actual structural reduction, essential for dense model size.
- **ChrF:** Primary text quality metric, character-n-gram based, suited for morphologically rich scripts.
- **BLEU:** Standard machine translation reference.
- **ASR-ChrF and ASR-BLEU:** For speech-to-speech output, evaluating the transcribed generated waveform against reference text to validate the audio path and T2U module.
- **RTF (Real-Time Factor):** Inference time divided by audio duration. Lower is better for interactive potential.
- **VRAM:** Tracked for practical local deployability.

6 Results

Table 1: Phase-by-phase summary

Phase / Method	Parameters	ChrF	RTF
P0: Baseline S2ST	1805.5M	46.49	0.2455
P1: Five-language vocab pruning	1566.6M	41.74	0.2435
P2: Speech encoder 24 to 16 layers	1373.1M	38.97	0.1617
P3: T2U merge	1331.2M	38.47	0.1646
P4: Speech encoder 16 to 14 layers	1282.8M	35.74	0.1635
P5: Text decoder 24 to 14 layers	1030.9M	25.32	0.1881
P6: KD recovery	1030.0M	33.07	0.1484
P7: Final hybrid adapter recovery	1087.9M	33.73	0.1336

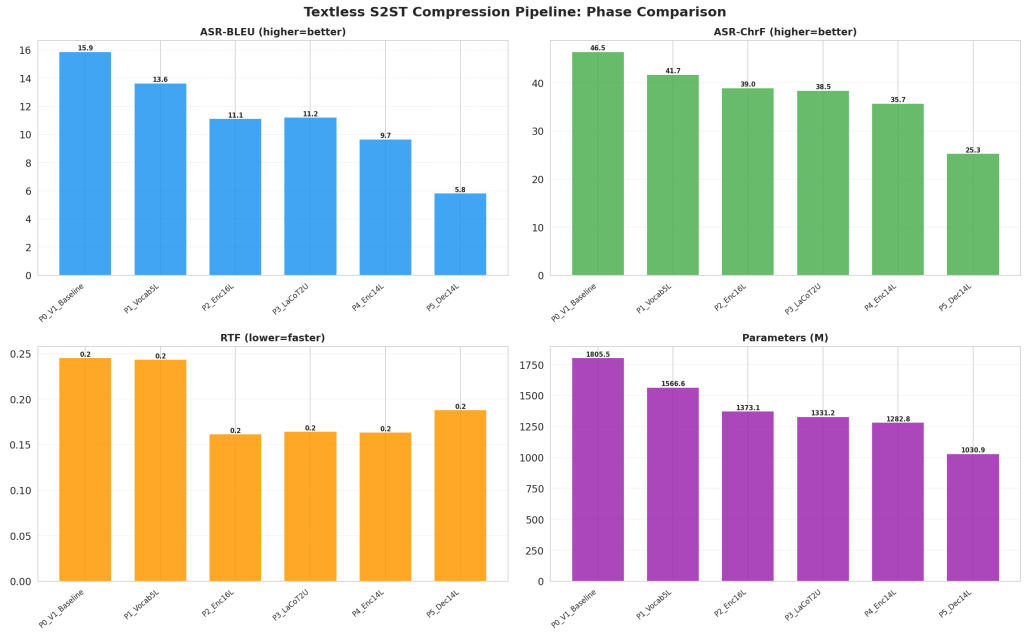

Figure 4: Overall quality comparison across different pruning and recovery phases.

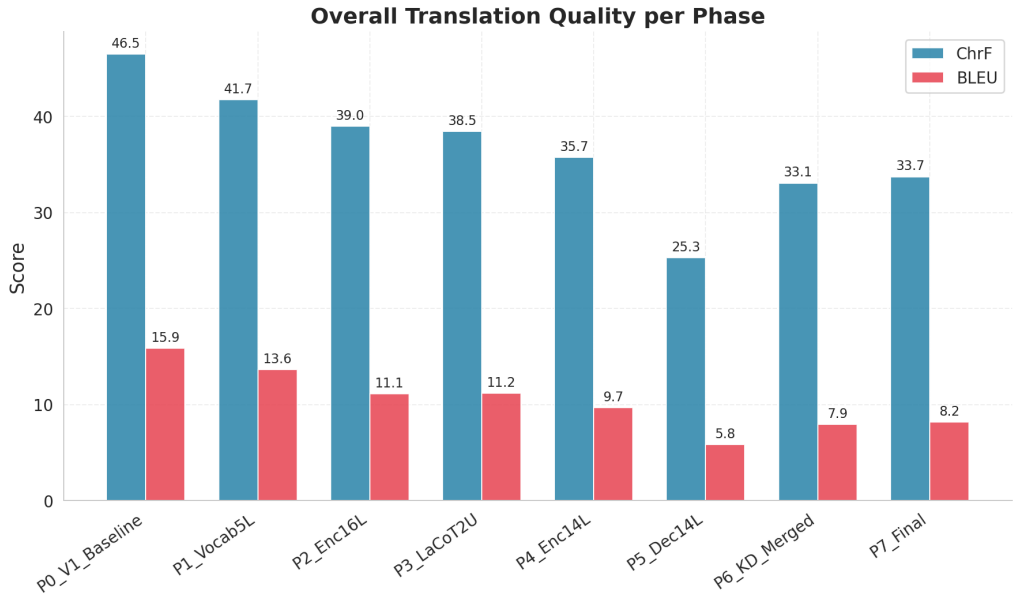

Figure 5: Detailed breakdown of overall quality metrics showing the impact of successive structural reductions.

Table 2: Final per-pair results

Pair	**ChrF**	**BLEU**	**RTF**
arb to eng	39.62	10.42	0.1241
ben to eng	34.40	6.11	0.0969
cmn to eng	35.62	7.33	0.1288
eng to arb	32.96	6.51	0.1144
eng to ben	39.32	8.49	0.1409
eng to cmn	5.61	2.29	0.2529
eng to hin	44.68	15.41	0.1014
hin to eng	37.65	8.75	0.1094

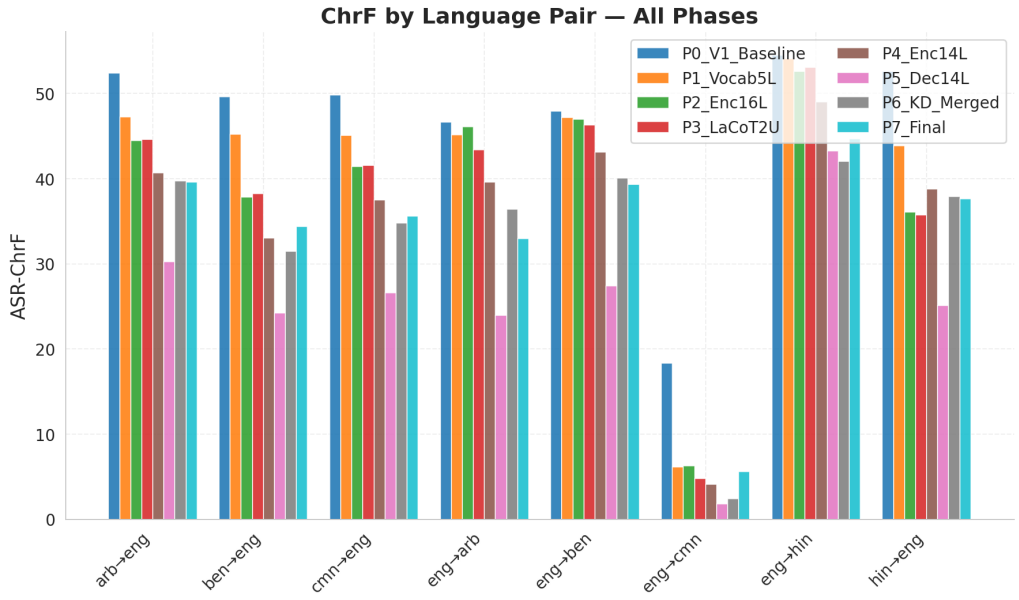

Figure 6: ChrF scores broken down by individual language pairs.

NOTE: Due to the limitation of the ChrF and Bleu calculation algorithm we used, we cannot rely on it for the Chinese translations. That's why Chinese as a Target language in our benchmarks got near zero marks. But the actual quality of Chinese translation is not that bad in our model.

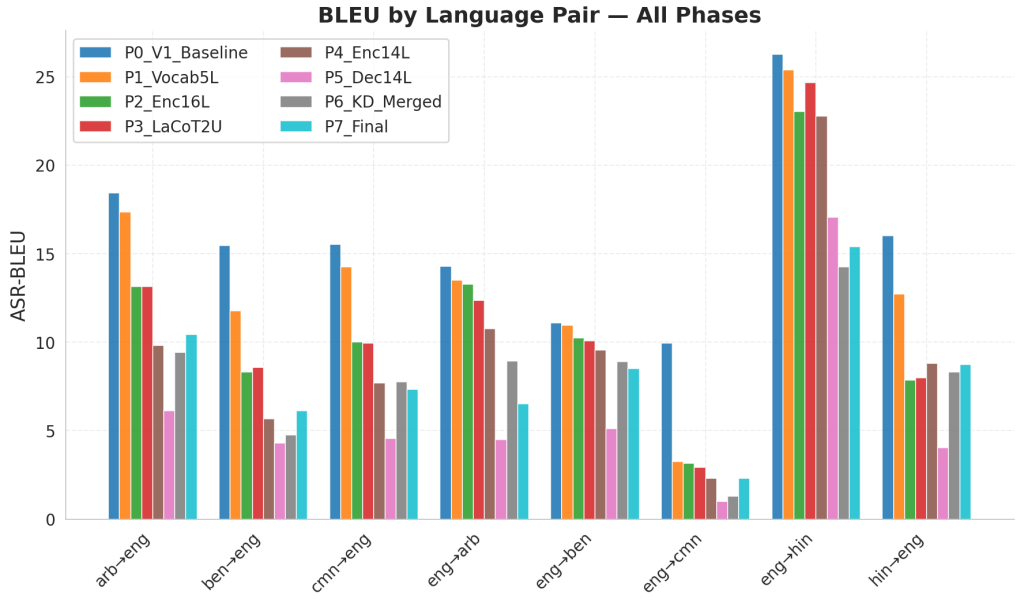

Figure 7: BLEU scores broken down by individual language pairs.

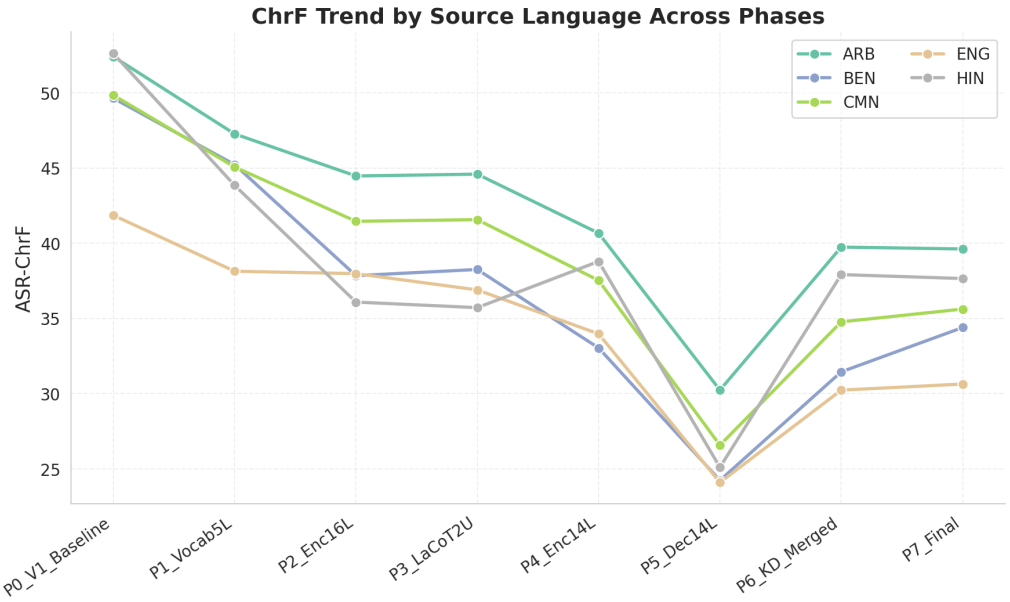

Figure 8: Performance trends grouped by source language.

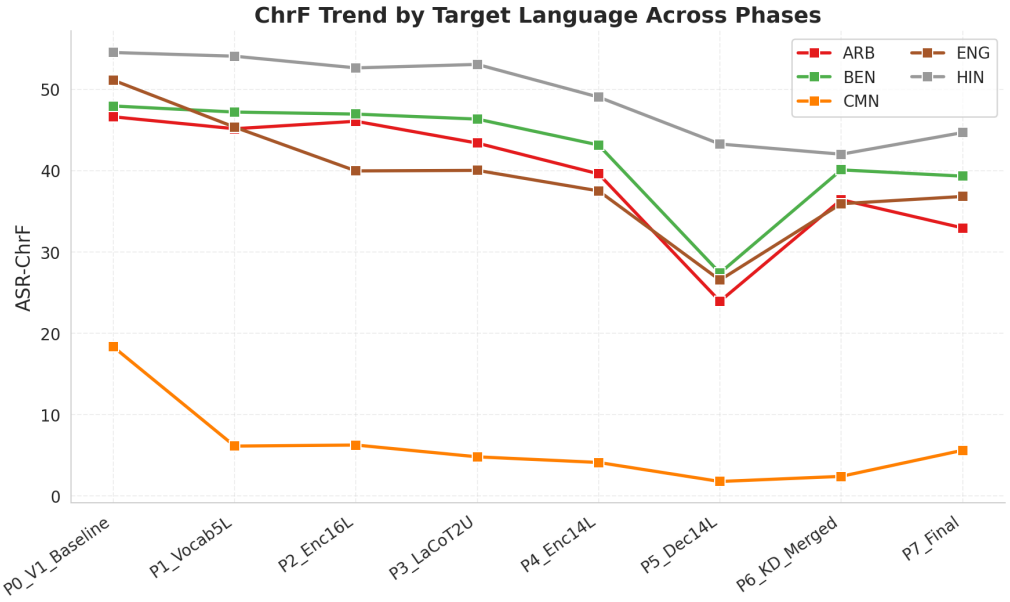

Figure 9: Performance trends grouped by target language.

Figure 10: Trade-off analysis: Model parameter size versus translation quality.

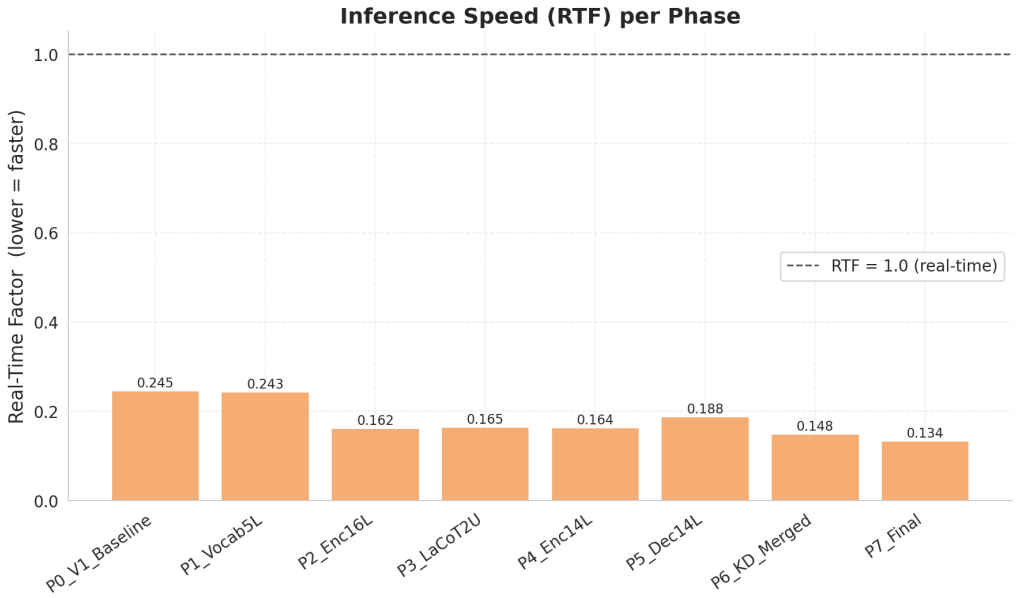

Figure 11: Inference Real-Time Factor (RTF) comparisons across the pipeline.

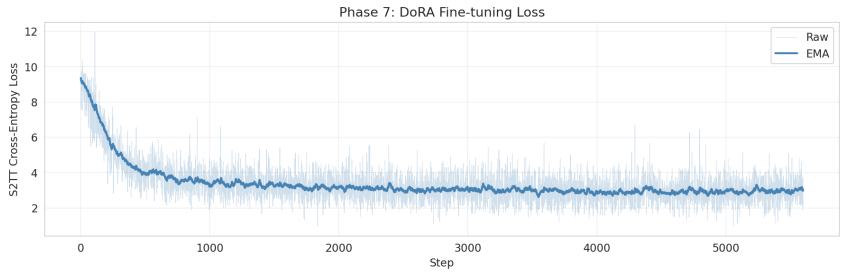

Figure 12: Training loss curve during the Phase 7 final hybrid adapter recovery.

The final model trades quality for size and speed, crossing the practical deployment threshold on local hardware with a 1.84x RTF speedup. Ablation tests demonstrated that width pruning (FLAP) and extreme compression targets (sub-500M) led to catastrophic generation collapse, highlighting that the text decoder serves as the core multilingual sequence modeling engine. The final model trades quality for size and speed, crossing the practical deployment threshold on local hardware with a 1.84x RTF speedup. Ablation tests demonstrated that width pruning (FLAP) and extreme compression targets (sub-500M) led to catastrophic generation collapse, highlighting that the text decoder serves as the core multilingual sequence modeling engine.

7 Conclusion

This project successfully compressed, recovered, and deployed SeamlessM4T v2 Large for multilingual speech-to-speech translation. The final selected model reduces parameters from 1.8055B to 1.0879B, improves RTF from 0.2455 to 0.1336, and runs on a 4 GB-class laptop GPU. It also powers a duplex browser application with streaming microphone input, boundary detection, translated audio playback, and barge-in handling. Where earlier, the model safetensor size was around 9.6 GB, and ours version is currently 1.9GB only which is 1/4th the size of the actual base model.

The most important research findings demonstrate that language-scoped vocabulary pruning is highly effective, BI-guided depth pruning requires task metric validation, T2U components require ASR-based metrics, and the text decoder is the main limiting factor for extreme compression.

We also aim to convert this full project to a locally usable Android or iOS app version within a short period of time to make it an industry standard product.

8 Mandatory Appendix - 1

8.1 Link to Code / Github Repository

https://github.com/rayedriasat/Pruning-SeamlessM4T

8.2 Reproducible Instructions and Prompt for an Agent

Below is the strict prompt designed for an autonomous coding agent (like OpenCode) to recreate this complete pipeline from scratch.

```
# From-Scratch Reproduction Prompt for an Agent Like OpenCode

You are an autonomous research/coding agent. Your task is to reproduce a final
compressed SeamlessM4T v2 multilingual speech-to-speech translation system and
its duplex web application from scratch.

Important assumption: you do not have access to the original project notebooks,
saved notebook outputs, or hidden checkpoint files. You only know the target
methodology and expected outcomes described in this prompt. You must implement
the pipeline yourself, create clean scripts/notebooks, run experiments, save
checkpoints, and report deviations.

## Goal

Build a structurally compressed and recovered version of
‘facebook/seamless-m4t-v2-large‘ for multilingual speech-to-speech translation
over five languages:

- English: ‘eng‘, FLEURS ‘en_us‘
- Bengali: ‘ben‘, FLEURS ‘bn_in‘
- Mandarin Chinese: ‘cmn‘, FLEURS ‘cmn_hans_cn‘
- Arabic: ‘arb‘, FLEURS ‘ar_eg‘
- Hindi: ‘hin‘, FLEURS ‘hi_in‘

The final target model should be close to:

- 1.09B parameters
- 14 speech encoder layers
- 14 text decoder layers
- conservative T2U compression/merge
- pruned vocabulary for the five-language task
- recovered with teacher-student KD and LoRA/DoRA-style adapters
- capable of local inference and browser-based duplex speech translation

Expected final benchmark ballpark:

| Metric | Baseline target | Final target |
| --- | ---: | ---: |
```

Parameters	about 1805.5M	about 1088M
ChrF	about 46.5	about 33-35
BLEU	about 15.9	about 8-9
RTF	about 0.245	about 0.13-0.15

Small deviations are acceptable if you document them. Do not force the model below 500M if quality collapses.

Deliverables

Create the following deliverables:

1. ‘reproduce/requirements.txt‘ or ‘pyproject.toml‘
2. ‘reproduce/prepare_data.py‘
3. ‘reproduce/model_utils.py‘
4. ‘reproduce/metrics.py‘
5. ‘reproduce/prune_vocab.py‘
6. ‘reproduce/prune_layers.py‘
7. ‘reproduce/recover_kd.py‘
8. ‘reproduce/benchmark.py‘
9. ‘reproduce/local_inference.py‘
10. ‘webapp/‘ with a FastAPI/WebSocket backend and browser frontend
11. ‘artifacts/results/final_results.md‘
12. ‘artifacts/results/phase_summary.csv‘
13. Saved model folders for each phase, or checkpoint instructions if storage is external

You may implement as scripts, notebooks, or both. Prefer scripts for reproducibility.

Hardware and Environment

Use a CUDA GPU environment. Recommended:

- Training/pruning: two NVIDIA T4 GPUs or better, 15 GB VRAM each.
- Local inference validation: any CUDA GPU with 4 GB or more VRAM.

Install:

```
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cu121
pip install transformers datasets accelerate peft sentencepiece sacrebleu evaluate
librosa soundfile pandas pyarrow numpy scipy matplotlib seaborn fastapi uvicorn
websockets python-multipart safetensors
```

If your CUDA version differs, install the correct PyTorch wheel for your environment.

Data Preparation

Use FLEURS from HuggingFace Datasets.

Create train/eval data for these language pairs:

```
EVAL_PAIRS = [
    ("ar_eg", "en_us", "arb", "eng"),
    ("bn_in", "en_us", "ben", "eng"),
    ("cmn_hans_cn", "en_us", "cmn", "eng"),
    ("hi_in", "en_us", "hin", "eng"),
    ("en_us", "ar_eg", "eng", "arb"),
    ("en_us", "bn_in", "eng", "ben"),
    ("en_us", "cmn_hans_cn", "eng", "cmn"),
    ("en_us", "hi_in", "eng", "hin"),
]
```

Use 25 test samples per pair for full benchmarking. For pruning candidate probes, use a smaller subset such as 5 samples per pair to control runtime.

For each paired sample, store:

- `id`
- source waveform as float32 16 kHz mono
- source language code
- target language code
- target transcription/reference text
- optionally source transcription text

Cache the processed data as Parquet or `.pt` files. Do not repeatedly download or decode audio inside every pruning loop.

Baseline Model

Load:

```
from transformers import SeamlessM4TProcessor, SeamlessM4Tv2ForSpeechToSpeech

processor = SeamlessM4TProcessor.from_pretrained("facebook/seamless-m4t-v2-large")
model = SeamlessM4Tv2ForSpeechToSpeech.from_pretrained(
    "facebook/seamless-m4t-v2-large",
    torch_dtype=torch.float16,
).cuda().eval()
```

Important: use `SeamlessM4Tv2ForSpeechToSpeech`. Do not count or prune the text encoder, because this class does not use it in the S2ST path.

Implement:

- ‘count_params(model)‘
- ‘print_component_breakdown(model)‘
- ‘sync_model_config(model)‘ to update layer counts after pruning
- ‘reindex_layer_idx(module)‘ for decoder/T2U attention layers after removing layers

Run a baseline benchmark over all 8 pairs:

- generate speech
- decode intermediate text if available
- transcribe generated speech with ASR when measuring ASR metrics
- compute ChrF/BLEU against reference text
- measure RTF

Save the baseline model summary.

Metrics

Use SacreBLEU:

```
from sacrebleu.metrics import BLEU, CHRF
bleu = BLEU(effective_order=True)
chrf = CHRF()
```

Implement:

- ‘compute_bleu(pred, ref)‘
- ‘compute_chrf(pred, ref)‘
- ‘run_s2st(model, processor, wav, tgt_lang)‘
- ‘run_s2tt_text(model, processor, wav, tgt_lang)‘ if available/needed
- ‘benchmark_s2st_asr(model, samples, label)‘

For T2U/audio-path decisions, use ASR-ChrF:

1. Generate output audio.
2. Transcribe it with a multilingual ASR model.
3. Compare ASR transcript to target reference with ChrF.

Use MMS ASR for multilingual support. If implementation time is limited, benchmark T2U on non-English target directions first, especially Bengali.

Phase 1: Five-Language Vocabulary Pruning

Implement vocabulary trimming:

1. Define target languages: ‘eng‘, ‘ben‘, ‘cmn‘, ‘arb‘, ‘hin‘.
2. Scan FLEURS train text for all target languages.
3. Collect token IDs used by the tokenizer.

```
4. Always keep:
   - special tokens
   - language tokens
   - padding/eos/bos/unk tokens
   - any generation config token IDs
5. Build `keep_ids`.
6. Rebuild the shared embedding:
   - new shape `[len(keep_ids), hidden_size]`
   - copy old rows by `keep_ids`
7. Rebuild/tie:
   - `model.shared`
   - `model.text_decoder.embed_tokens`
   - `model.lm_head`
8. Update `model.config.vocab_size`.
9. Update language token mappings in generation config.
10. Save a tensor named `_vocab_remap_to_old = torch.tensor(keep_ids)`.

Important decoding requirement:

When decoding intermediate IDs from the pruned model, remap them to original
tokenizer IDs:

ids_old = model._vocab_remap_to_old[ids_new]
text = processor.batch_decode(ids_old, skip_special_tokens=True)

Benchmark and save Phase 1.

Expected ballpark:

- parameters around 1.56B
- noticeable but acceptable quality drop

## Phase 2: Speech Encoder Pruning, 24 to 16 Layers

Implement Block Influence:

BI(layer) = 1 - cosine_similarity(layer_input, layer_output).mean()

Procedure:

1. Find the speech encoder layer `ModuleList`.
2. Register forward hooks on every layer.
3. Run 50 calibration samples through the speech encoder.
4. Compute BI score per layer.
5. Protect layers `{0, n//2, n-1}`.
6. Iteratively remove 8 layers:
   - candidate set: non-protected layers
   - optionally pre-filter to lowest 50 percent BI scores
```

```
   - for each candidate, temporarily remove it
   - sync config
   - run small multilingual ChrF probe
   - restore model
   - choose the candidate with the highest probe score
   - permanently remove it
   - save checkpoint

After pruning:

- speech encoder should have 16 layers
- sync all config layer counts
- save model and benchmark

Expected ballpark:

- parameters around 1.37B
- RTF around 0.16

## Phase 3: Conservative T2U Layer Merge

The T2U path affects output audio and is sensitive. Do not aggressively delete
T2U layers unless ASR metrics confirm it.

Implement a conservative merge:

1. Find T2U encoder and decoder layer stacks.
2. For adjacent candidate layers, create a merged candidate by
   averaging/interpolating compatible parameters:

merged = alpha * layer_i + (1 - alpha) * layer_j

3. Evaluate output similarity on calibration hidden states.
4. Merge only if similarity is high enough, e.g. cosine similarity above 0.85.
5. Limit removals per T2U stack.
6. Re-index T2U attention `layer_idx`.
7. Sync T2U config fields.

Benchmark after merge.

Expected ballpark:

- parameters around 1.33B
- small additional quality loss

## Phase 4: Additional Speech Encoder Pruning, 16 to 14 Layers

Repeat the Phase 2 BI-guided method, but remove only 2 additional speech encoder
```

```
layers.

Benchmark and save.

Expected ballpark:

- parameters around 1.28B
- quality drops more noticeably than Phase 2

## Phase 5: Text Decoder Pruning, 24 to 14 Layers

The text decoder is critical. Do not prune below 14 layers for the final target.

Procedure:

1. Find `model.text_decoder.layers`.
2. Compute decoder BI using hooks during generation.
3. Protect `{0, n//2, n-1}`.
4. Iteratively remove 10 layers:
   - use BI pre-filtering
   - evaluate small multilingual ChrF probe
   - remove least damaging layer
   - re-index decoder attention layer indices
   - sync config
   - checkpoint after every removal
5. Save model as the pruned student.

Expected ballpark:

- parameters around 1.03B
- quality cliff is expected
- ChrF may fall into the mid-20s

Do not run FLAP/width pruning in the final path. Previous evidence showed
catastrophic collapse after depth pruning.

## Phase 6: Teacher-Student KD Recovery

Load:

- teacher: full `facebook/seamless-m4t-v2-large`, frozen
- student: Phase 5 pruned model, trainable through adapters

Recommended GPU placement:

- student on `cuda:0`
- teacher on `cuda:1`
```

Attach LoRA/DoRA-style adapters to key student modules:

- text decoder self-attention q/v/out projections
- text decoder cross-attention q/v/out projections
- text decoder FFN layers
- selected T2U layers if memory permits

Use mixed precision carefully:

- base weights fp16
- trainable adapter params fp32 if using GradScaler
- freeze all non-adapter parameters

Loss:

```
loss = (1 - alpha) * CE(student_logits, labels) + alpha * KL(student_logits,
       teacher_topk_logits)
```

Use ‘alpha‘ around 0.15 as a starting point.

Sparse teacher KD:

1. Run teacher forward without gradients.
2. Take top-k teacher logits, e.g. k=128 or 256.
3. Remap teacher token IDs to student token IDs using the vocabulary map.
4. Drop unmapped teacher tokens.
5. Renormalize teacher probabilities over mapped tokens.
6. Gather student logits at mapped token positions.
7. Compute KL at temperature around 3.0.

Checkpoint:

- adapter weights
- optimizer state
- best validation ChrF
- step count

Train until validation improves and stabilizes. Save merged model.

Expected ballpark after Phase 6:

- ChrF around 33
- BLEU around 8
- RTF around 0.15

Phase 7: Final Adapter Recovery and Merge

Run a final recovery pass if Phase 6 quality is still low.

Recommended:

- load Phase 6 merged model
- attach broader hybrid adapters
- keep speech encoder frozen unless evidence suggests otherwise
- prioritize text decoder and T2U/text-unit path
- train for several epochs over FLEURS multilingual data
- evaluate every fixed number of steps on the 8-pair validation subset
- save best checkpoint
- merge adapters into base weights

Final save format must be directly loadable with:

SeamlessM4TProcessor.from_pretrained(final_model_dir)
SeamlessM4Tv2ForSpeechToSpeech.from_pretrained(final_model_dir)

Also save:

- `_custom_state.pt` containing `_vocab_remap_to_old`
- generation config
- tokenizer/processor files
- model config with correct layer counts

Final Benchmark

Benchmark the final merged model on:

- 8 language pairs
- 25 samples per pair
- 200 total samples

Report parameters, ChrF, BLEU, RTF, per-pair metrics, audio samples, GPU usage, and failure cases.

Local Inference Tool

Create a local inference script or notebook that:

1. Loads the final model from a local folder.
2. Repairs stale config layer counts from actual weight keys if needed.
3. Loads `_custom_state.pt` and attaches `_vocab_remap_to_old`.
4. Accepts a WAV file or FLEURS sample.
5. Runs S2ST.
6. Saves output WAV.
7. Prints intermediate decoded text if available.
8. Prints GPU memory usage.

Target local validation:

- RTX 3050 Laptop GPU or similar 4 GB-class GPU
- model memory around 2-3 GB fp16

Duplex Web App

Build a browser app that uses the final model interactively.

Backend:

- FastAPI
- WebSocket endpoint `/ws`
- health endpoint `/health`
- static frontend serving
- load final SeamlessM4T model once at startup
- load a boundary/VAD adapter if available
- use an inference lock so boundary checks and translation do not overlap on GPU

Frontend:

- `index.html`
- `app.js`
- `worklet.js`
- capture mic at 16 kHz
- use AudioWorklet to emit int16 PCM frames
- stream binary frames over WebSocket
- support push-to-talk and always-listen
- schedule translated output audio
- flush scheduled playback on barge-in

Session behavior logic... (Follow the requirements as discussed in literature).

Boundary detector:

Use a 3-layer MLP over SeamlessM4T speech encoder hidden states:

```
Linear(1024, 512)
LayerNorm
ReLU
Dropout
Linear(512, 256)
LayerNorm
ReLU
Dropout
Linear(256, 1)
Sigmoid
```

Failure Boundaries to Respect

Do not pursue these as the final path:

1. Do not force a sub-500M model by pruning the text decoder to 6 layers. It is likely to collapse.
2. Do not remove the text decoder entirely or replace it with a simple CIF shortcut.
3. Do not use text-only metrics to decide T2U pruning.
4. Do not apply FLAP width pruning after heavy depth pruning.
5. Do not forget to sync config layer counts and attention ‘layer_idx‘.

Final Report I will be required from the codes are below:

At the end, produce ‘artifacts/results/final_results.md‘ with phase-by-phase results, parameters, deviations, application status, and recommendations.